

Smart Wagon — MQTT Interface Reference

Every uplink and downlink message, with how to send each command and how to verify it landed · gateway firmware 1.2.0 · protocol pv=1

1. Connection

Every value below is compiled into `GATEWAY/src/app_config.h` and derived from a single constant, `WAGON_NUMBER`. Change that one define and the client id, both topic keys, the BLE isolation group and the per-wagon crypto key all follow.

Setting	Value	Source
Broker	mqtt.macnman.com : 1883	MQTT_HOST / MQTT_PORT
Client id	GW-31054312345	GW_ID = "GW-" + WAGON_NUMBER
Username / password	empty (anonymous)	MQTT_USERNAME
Session	persistent, <code>clean_session = 0</code>	so downlinks queue while the modem is off
QoS	1, both directions	cloud de-dupes on (gw, seq)
Topic root	smartwagon/v1	TOPIC_ROOT
Protocol version	<code>pv = 1</code>	PROTO_PV

1.1 Topic map

Pattern: `smartwagon/v1/{wgn}/{gw}/{dir}/{kind}`, where `{wgn}` is the wagon number and `{gw}` is the gateway id.

Direction	Topic	Published by
uplink	smartwagon/v1/31054312345/GW-31054312345/gateway	gateway - scheduled heartbeat
uplink	.../up/alarm	gateway - safety-critical alarms
uplink	.../up/event	gateway - state-change events
uplink	.../up/resp	gateway - reply to every command
downlink	.../dn/cmd	you - the only topic the gateway subscribes to

The gateway subscribes exactly once, in `ec200.c`: `AT+QMTSUB=0,1,"<root>/<wgn>/<gw>/dn/cmd",1`. There is no wildcard and no broadcast topic - each gateway is addressed individually.

2. THE MOST IMPORTANT THING: when does a command arrive?

The gateway is not always online. Its modem is powered OFF between reports to meet the battery budget. Commands are drained only **after an uplink** (`drain_commands()`, called right after the heartbeat is published). So a command you publish now is delivered at the gateway's **next scheduled report**:

- wagon **moving**: within **10 minutes**
- wagon **parked**: within **12 hours**

This is why the session must be persistent (`clean_session=0`) and QoS 1 - the broker holds the message until the gateway wakes. If you publish with QoS 0, or with a clean session, the command is simply lost.

Nothing is broken if you do not see a `resp` within seconds. Subscribe to `up/resp` and wait for the next report window.

3. How to send a downlink

Open two terminals. The first watches everything the wagon says; the second sends the command.

Terminal 1 - watch all uplinks

```
mosquitto_sub -h mqtt.macnman.com -p 1883 -v \ -t 'smartwagon/v1/31054312345/GW-31054312345/up/#'
```

Terminal 2 - send the command

```
mosquitto_pub -h mqtt.macnman.com -p 1883 -q 1 \ -t 'smartwagon/v1/31054312345/GW-31054312345/dn/cmd' \ -m '{"cmd":"get_config","cid":"c-0001"}'
```

cid is yours to choose and you should always set it. It is echoed back verbatim in the response, and it is the only way to match a reply to the request you sent - responses all arrive on the same topic, possibly out of order, possibly hours later. Use something unique per request.

4. Downlink command reference

All 12 commands below are dispatched in `handle_command()`. Anything else returns `{"res":"err", "ec":"UNSUPPORTED"}`.

4.1 Command summary

cmd	Purpose	Applies
get_config	Read back the whole live config	immediately
get_status	Force a heartbeat now	immediately
set_threshold	Gateway impact-g, or a node's threshold	immediately / queued
set_interval	Heartbeat cadence, moving and parked	next wake
set_gnss	Enable, fix timeout, constellation	immediately
time_sync	Set the software clock from epoch	immediately
ota_start	Gateway firmware update	reboots
ota_status	Poll update progress	immediately
get_history	Ask for buffered records	triggers replay
pull_data	On-demand sensor snapshot	immediately
reboot	Cold restart	after the resp
provision_node	Map a node to this wagon	rejected - roster is compile-time

4.2 set_threshold - GATEWAY impact threshold

```
{"cmd":"set_threshold","cid":"c-1001","param":"impact_g","value":4.0}
```

Field	Type	Accepted range
param	string	must be "impact_g"
value	number (g)	> 0.5 and ≤ 16.0 - anything else returns BAD_PARAM

Response: `{"res":"ok", "pl":{"applied":true}}`. Stored as milli-g in `g_cfg.impact_mg` and written to FRAM. Live on the very next comparison - no reboot.

Verify:

```
mosquitto_pub -h mqtt.macnman.com -q 1 \ -t 'smartwagon/v1/31054312345/GW-31054312345/dn/cmd' \ -m '{"cmd":"get_config","cid":"c-1002"}'
```

The response carries `"impact_mg":4000`. That read-back is the authoritative check - it is printed from the live struct, not from what you sent.

4.3 set_threshold - SUB-NODE threshold

```
{"cmd":"set_threshold","cid":"c-1003","node":3,"value":850,"hyst":50}
```

Field	Type	Notes
node	int	node id from the roster (<code>nodes.c</code>), 0.. <code>SW_MAX_NODES-1</code>

value	int	in the node's own units - for a bearing that is tenths of °C, so 850 = 85.0 °C
hyst	int	release deadband, same units. 50 = alarm clears below 80.0 °C

Always send hyst. If you omit it the gateway sends **0**, and the node applies it - deleting the deadband rather than keeping the old value. A node with zero hysteresis chatters in and out of alarm at the trip point, which switches it between 30 s and 1 s advertising (~134 µA vs ~1800 µA) and floods the escalation path. Nothing in the response warns you.

Response: {"res":"ok", "pl":{"applied":false, "state":"queued"}}.

"queued" is not "applied". The node is only reachable during its own 4 s config window, which opens every ~10 minutes. The gateway holds the value and delivers it over BLE when the observer next sees that node's window. Total worst case from your publish: up to 12 h (gateway uplink) + ~10 min (node window) + one node cycle.

Two error cases to expect:

ec	Meaning
NOT_FOUND	The gateway has never heard this node, so it does not know the node type - and the type is authenticated inside the encrypted frame, so the command cannot even be built. Check the node is powered and advertising.
BAD_PARAM	node outside 0..SW_MAX_NODES-1, or value missing.

Verify (this is the only real confirmation): wait for the next heartbeat and read the `sens[]` entry for that node. The node's own uplink reflects its live threshold - if the write had failed authentication, addressing or the replay check, the node would have dropped it silently and the reading would still behave on the old value. There is no acknowledgement from the node itself; the behaviour of its subsequent alarms is the proof.

4.4 set_interval

```
{"cmd":"set_interval","cid":"c-1004","moving_s":600,"idle_s":43200}
```

Field	Accepted range	Default
moving_s	30 .. 86400 s	600 (10 min)
idle_s	60 .. 604800 s	43200 (12 h)

Either or both may be given. If neither is present **or** both are out of range, nothing is saved and you get BAD_PARAM - the gateway only writes FRAM on a real change. Takes effect at the next `arm_schedule()`. **Verify with `get_config` (`run_s` / `stop_s`).**

Shortening `idle_s` is the fastest way to make a parked wagon responsive while you are testing - a 12 h command latency is painful on the bench. Remember to put it back.

4.5 set_gnss

```
{"cmd":"set_gnss","cid":"c-1005","enable":true,"fix_timeout_s":90,"constellation":"navic_gps"}
```

Field	Values
enable	true / false. False means "report last-known position and skip the fix" - saves the GNSS rail entirely.
fix_timeout_s	10 .. 300 s
constellation	auto navic gps navic_gps

Pushed to the LC29H immediately via PAIR066/PAIR513 (persisted in the module). Always answers ok. **Verify with `get_config`:** `gnss_enable`, `gnss_constel` (0 auto / 1 navic / 2 gps / 3 navic_gps), `gnss_timeout_s`.

4.6 get_config - the read-back you will use most

```
{"cmd":"get_config","cid":"c-1006"}
```

```
{"res":"ok","pl":{"run_s":600,"stop_s":43200,"impact_mg":4000,"gnss_enable":1,"gnss_constel":3,"gnss_timeout_s":90,"nodes":19,"fw":"1.2.0"}}
```

Printed straight from the live `g_cfg` struct, so it is the truth about what the gateway is running - not a copy of what you asked for.

4.7 get_status and pull_data

```
{"cmd":"get_status","cid":"c-1007"} {"cmd":"pull_data","cid":"c-1008"}
```

Both force a full heartbeat immediately, which is how you get a fresh `sens[]` array on demand rather than waiting for the schedule. `pull_data` additionally reports how many nodes are provisioned. Use either to confirm a node threshold landed.

4.8 time_sync

```
{"cmd":"time_sync","cid":"c-1009","epoch":1718971412}
```

Sets the software clock. `epoch` must be > 0 or you get `BAD_PARAM`. Normally unnecessary - the clock is disciplined from GNSS UTC - but useful when GNSS has never had a fix and buffered records would otherwise carry a meaningless `ts`.

4.9 ota_start / ota_status - GATEWAY ONLY

```
{"cmd":"ota_start","cid":"c-1010","url":"https://host/gw-1.3.0.bin","ver":"1.3.0","size":327680}
```

Downloads over HTTP through the modem straight into the MCUboot secondary slot (never buffered in RAM), then reboots to apply. MCUboot verifies the signature; if the new image does not confirm itself after a healthy cycle it is **reverted automatically** on the next reset.

```
{"cmd":"ota_status","cid":"c-1011"} -> {"res":"ok","pl":{"state":"downloading","pct":47,"ver":"1.3.0"}}
```

States: `idle`, `downloading`, `verify`, `ready`, `applying`, `error`.

Sub-node firmware update does not exist. An `ota_start` carrying a `node` field is rejected with `{"res":"err","ec":"UNSUPPORTED","pl":{"info":"node OTA not supported"}}`. This is deliberate: without the check, a node-targeted update would flash the GATEWAY with node firmware. The only thing a node ever accepts over the air is a threshold.

Verify: after the reboot, `get_config` returns the new `fw` string. If it still shows the old version, MCUboot reverted - the image failed to confirm.

4.10 get_history

```
{"cmd":"get_history","cid":"c-1012","from_seq":20800}
```

Returns the **count** of buffered records, then triggers a replay - the records themselves arrive as normal uplinks on their own topics, republished with their **original** `seq` and `ts` so cloud de-dup still works. They are not embedded in the response: each is up to ~640 B and would overflow the modem's publish buffer.

```
{"res":"ok","pl":{"from_seq":20800,"count":37,"mode":"push","info":"records replay on reconnect with original seq/ts"}}
```

4.11 reboot

```
{"cmd":"reboot","cid":"c-1013"}
```

Publishes the response first, waits 500 ms, then cold-restarts - so you always learn it was accepted before the gateway disappears.

4.12 provision_node - rejected by design

```
{"cmd":"provision_node","cid":"c-1014","node":7,"typ":"bearing"} -> {"res":"err","ec":"UNSUPPORTED","pl":{"info":"roster is compile-time (nodes.c)"}}
```

The roster is a compile-time table because `node_id` must agree with the `NODE_ID` built into each sub-node's firmware. A runtime remap would let the gateway and the node silently disagree about which wheel a reading belongs to.

5. Response format (up/resp)

```
{"mt":"resp","pv":1,"seq":20903,"gw":"GW-31054312345","wgn":"31054312345","ts":1718971500,"loc":{"...},"spd":0,"pwr":{"...},"d":{"cid":"c-1001","cmd":"set_threshold","res":"ok","ec":null,"pl":{"applied":true}}}
```

Field	Meaning
cid	echoed verbatim from your command - match on this
cmd	echoed command name
res	ok err
ec	error code or null: BAD_PARAM, NOT_FOUND, UNSUPPORTED, BUSY
pl	command-specific payload. applied:true = in force now; state:"queued" = accepted but not yet delivered

6. Uplink messages

6.1 Common envelope

Every uplink - hb, alarm, event, resp - starts with the same envelope and differs only in d.

Field	Meaning
mt	message type: hb alarm event resp
pv	protocol version, always 1
seq	monotonic per gateway, persisted in FRAM - survives reset so (gw,seq) de-dup keeps working
gw / wgn	gateway id / wagon number
ts	UTC epoch seconds at the moment of capture, not of publication
loc	lat, lon, cep (CEP50 in m, derived from HDOP; 99 = no usable fix), sys (constellations actually seen)
spd	speed km/h
pwr	compact power block: src, soc, sol, bkp
d	the type-specific payload

A record replayed from the offline archive keeps its **original** seq and ts. So a burst of old timestamps after a coverage gap is correct behaviour, not a fault - order by ts, de-dup on (gw, seq).

6.2 Heartbeat (up/hb)

```
{
  "mt": "hb",
  "pv": 1,
  "seq": 20871,
  "gw": "GW-31054312345",
  "wgn": "31054312345",
  "ts": 1718971412,
  "loc": {
    "lat": 18.519900,
    "lon": 73.860120,
    "cep": 4,
    "sys": ["gps"]
  },
  "spd": 54,
  "pwr": {
    "src": "lto",
    "soc": 81,
    "sol": "off",
    "bkp": "ok"
  },
  "d": {
    "mode": "running",
    "mot": {
      "spd": 54,
      "hdg": 271
    },
    "env": {
      "t": 31.4,
      "h": 48
    },
    "gnss": {
      "sys": ["gps"],
      "fix": "3d",
      "nsat": 11,
      "hdop": 0.8
    },
    "sens": [
      {
        "node": "NODE-BRG-03",
        "pos": "B1-A1-L",
        "typ": "bearing",
        "val": 62,
        "unit": "C",
        "t": 0,
        "bat": 97,
        "age": 12
      },
      {
        "node": "NODE-D00R-14",
        "pos": "B1D-L",
        "typ": "door",
        "miss": 1
      }
    ],
    "band": {
      "brg": ["G"],
      "whl": ["G"]
    },
    "pwr2": {
      "src": "lto",
      "lto": {
        "v": 7420,
        "soc": 81,
        "chg": "fast"
      },
      "bkp": {
        "ok": true,
        "rem": 18450
      },
      "gsm": {
        "cell": "0",
        "rssi": -71
      }
    },
    "buf": {
      "n": 0,
      "old": 0,
      "new": 0,
      "up": 146,
      "fw": "1.2.0"
    }
  }
}
```

Field	Meaning
mode	running stopped - the DEBOUNCED motion state, not a raw sample
sens[]	one entry per provisioned node, always the full roster
sens[].age	seconds since that node's advert was last heard - the freshness of the reading
sens[].miss	1 = not heard . The node is reported as missing rather than silently omitted, so a dead node cannot look like a healthy roster
buf.n	records still queued in the NOR archive - non-zero means the wagon has been offline
up	uptime in hours

6.3 Alarm (up/alarm)

```
{ "mt": "alarm", ... , "d": { "code": "HOT_BEARING", "sev": "red", "node": "NODE-BRG-03",
"pos": "B1-A1-L", "val": 118, "unit": "C", "thr": 95, "st": "set", "aid": "a1-20872", "spd": 52 } }
```

Field	Meaning
code	see the enumeration below
sev	yellow/red for band alarms; warn/crit otherwise
node	node serial, or null for a gateway-level alarm
st	set on onset, clear on return to normal
aid	alarm instance id - the matching clear reuses it
spd	speed at trigger, preserved so a buffered alarm stays meaningful

Alarm codes (Protocol Rev.1 §3.2): DERAIL, HOT_BEARING, FLAT_WHEEL, DOOR_UNAUTH, HANDBRAKE_MOVING, OVERLOAD, TILT, TANK_OVERTEMP, IMPACT, SENSOR_FAULT, TAMPER, LOW_BATTERY, NODE_LOW_BATTERY.

FLAT_WHEEL is only raised above 15 km/h (§7.9). NODE_LOW_BATTERY is advisory (sev:warn) and rides the next heartbeat rather than waking the unit. TAMPER covers gateway or sensor removal/tampering, with the affected device in node.

6.4 Event (up/event)

```
{ "mt": "event", ... , "d": { "code": "TRAIN_START", "from": "stopped", "to": "running", "node": null, "trip": "trip-4471", "info": {} } }
```

Event codes: TRAIN_START, TRAIN_STOP, DOOR_OPEN, DOOR_CLOSE, HANDBRAKE_APPLIED, HANDBRAKE_RELEASED, LOAD_CHANGE, BAND_CHANGE, CHARGE_START, CHARGE_STOP, SRC_SWITCH, GEOFENCE_ENTER, GEOFENCE_EXIT, CONN_RESTORED, BOOT.

Events are edge-triggered in gwalarm.c, so a condition that stays true does not repeat every heartbeat. GEOFENCE_* is implemented but inactive (GEOFENCE_RADIUS_M = 0) - the protocol defines no provisioning command for a fence.

7. Worked example: change a bearing threshold end to end

- Watch.** `mosquitto_sub -h mqtt.macnman.com -v -t '.../up/#'`
- Find the node id.** Send `get_status`; read `sens[]` in the heartbeat that follows. NODE-BRG-03 means type bearing, id 3.
Send.

```
mosquitto_pub -h mqtt.macnman.com -q 1 \ -t 'smartwagon/v1/31054312345/GW-31054312345/dn/cmd' \ -m '{"cmd": "set_threshold", "cid": "brg3-a", "node": 3, "value": 800, "hyst": 50}'
```
- Wait for the resp** with "cid": "brg3-a". Up to 10 min moving, 12 h parked. Expect {"applied": false, "state": "queued"}.
- Wait ~10 more minutes** for the node's config window - no message is emitted when the BLE write lands.
- Confirm.** Send `pull_data` and check the node's behaviour against the new trip point. A bearing at 81 °C should now alarm; below 75 °C it should clear.

If nothing arrives at all: check in this order - (1) is the gateway online (any uplink at all in the last 12 h?); (2) did you publish with -q 1; (3) is the topic exactly right including the GW- prefix; (4) is cid unique so you are not matching an old reply.

8. Known gaps

Gap	Effect
Omitted hyst is sent as 0	Silently deletes the node's deadband. Always send it explicitly.
Queued node thresholds live in gateway RAM only	A gateway reset before delivery loses the command, after you were told "queued". Re-send if the gateway rebooted.
No node-level acknowledgement	The only confirmation a node accepted a threshold is its subsequent behaviour in sens[].
MQTT is plaintext on port 1883	No TLS. Commands are authenticated only by broker access. The BLE hop to the node IS authenticated (AES-CCM, per-wagon key).

Nothing has run on hardware	Every message shape here is from source, not from a captured trace.
-----------------------------	---

Ref: RDSO WD-35-MISC-2024 · SmartWagon Telemetry Protocol Rev.1 (pv=1). Generated from the firmware sources by DOCS/gen_mqtt_pdf.py - regenerate after changing main.c handle_command() or telemetry.c.